



Recibe una cálida:

¡Bienvenida!

Te estábamos esperando 😊 +

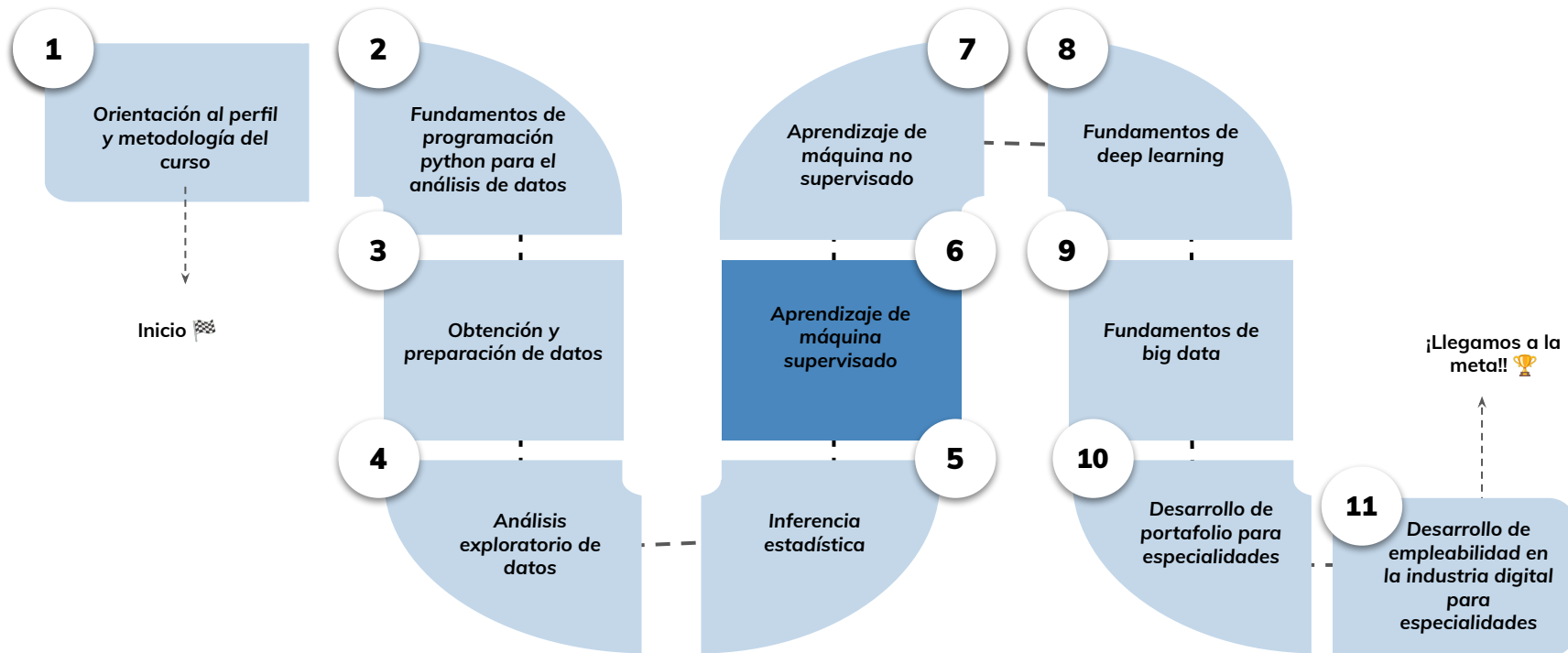


› Algoritmos de boosting- Parte II

Aprendizaje Esperado 8: Elaborar un modelo predictivo aplicando técnicas de boosting para resolver un problema de clasificación utilizando el lenguaje python.

Hoja de ruta

¿Cuáles skills conforman el programa? **Fundamentos de Ciencia de Datos**



Roadmap de lecciones

¿Cuáles **lecciones** estaremos estudiando en este módulo?

FUNDAMENTOS
DEL APRENDIZAJE
DE MÁQUINA

PREPROCESAMIENTO
O Y ESCALAMIENTO
DE DATOS

ALGORITMOS DE
CLASIFICACIÓN

OPTIMIZACIÓN DE
UN MODELO

1

2

3

4

5

6

7

8

NIVEL DE AJUSTE
DE UN MODELO Y
VALIDACIÓN
CRUZADA

REGRESIONES

MÉTRICAS DE
DESEMPEÑO DE UN
ALGORITMO DE
REGRESIÓN

ALGORITMOS DE
BOOSTING

Learning Path

¿Cuáles temas trabajaremos hoy?

8.

Técnicas avanzadas de Boosting

exploraremos las implementaciones más utilizadas en la industria para boosting, sus ventajas técnicas y casos de uso.

AdaBoost, XGBoost, LightGBM y CatBoost

Ajuste de hiperparámetros

Diferencias de arquitectura y regularización

Comparativa en velocidad, memoria y manejo de variables

Grid Search, Random Search y Optuna

Regularización, early stopping, learning rate



Objetivos de aprendizaje

¿Qué aprenderás?



- Comprender variantes avanzadas del boosting: AdaBoost, XGBoost, LightGBM, CatBoost.
- Identificar ventajas, limitaciones y casos de uso específicos.
- Implementar modelos de boosting para clasificación en Python.
- Ajustar hiperparámetros con Grid Search, Random Search y Optimización Bayesiana.
- Interpretar resultados mediante SHAP y gráficos de dependencia parcial.

Repaso clase anterior

¿Quedó alguna duda?

En la clase anterior trabajamos :

- Qué es el boosting y cómo funciona secuencialmente.
- Diferencias clave entre bagging y boosting.
- Implementación de `GradientBoostingClassifier` con Scikit-learn.
- Evaluación de desempeño y análisis de importancia de variables.

Algoritmos de boosting

Toma de Decisiones Informadas

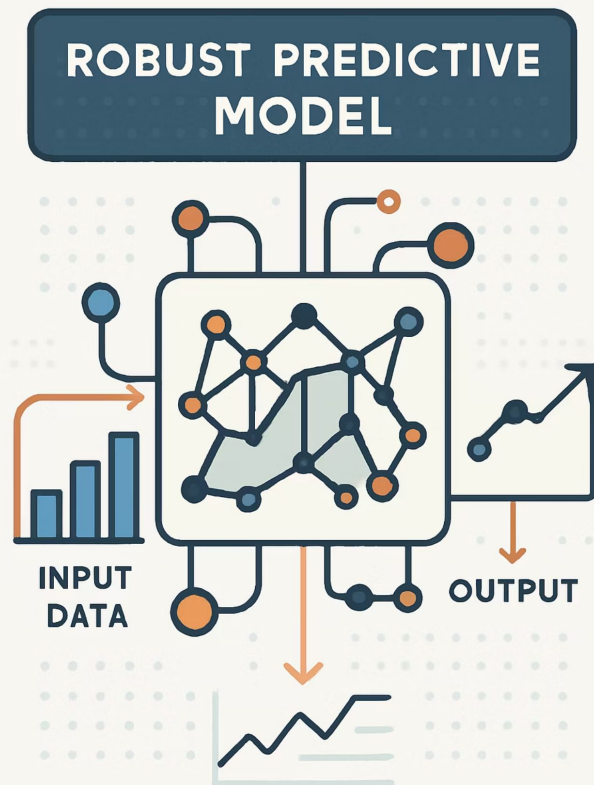
Al entender en profundidad las diferencias entre boosting y otros métodos ensemble como el bagging, es posible tomar decisiones más informadas al momento de seleccionar un enfoque para un proyecto determinado.

La implementación en Python, además, demuestra ser accesible, potente y altamente configurable, gracias al ecosistema de bibliotecas como Scikit-learn.

Consideraciones Finales

Es importante también conocer sus limitaciones, como el tiempo de entrenamiento o el riesgo de sobreajuste, para poder mitigarlas mediante prácticas como la regularización o la validación cruzada.

En definitiva, dominar Gradient Boosting permite construir modelos predictivos más precisos y robustos.

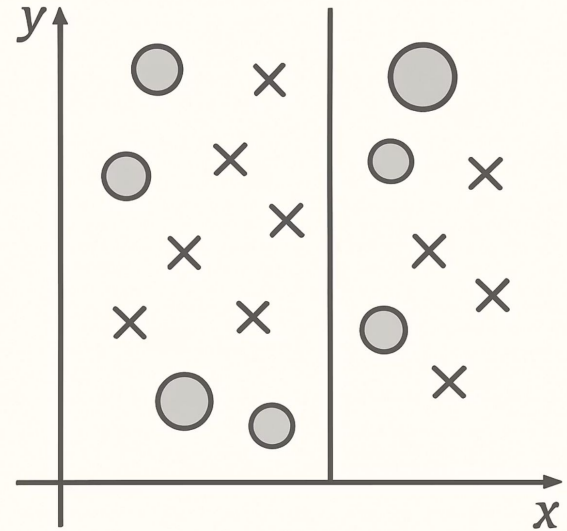


Profundizando en AdaBoost

AdaBoost (Adaptive Boosting) fue uno de los primeros algoritmos de boosting desarrollados y sigue siendo relevante en la actualidad.

Su principal característica es la forma en que ajusta los pesos de las observaciones: aumenta el peso de los ejemplos mal clasificados y disminuye el de los correctamente clasificados.

AdaBoost Algorithm with Weighted Samples



Funcionamiento de AdaBoost

Inicialización

Asigna pesos iguales a todas las observaciones del conjunto de entrenamiento.

Entrenamiento

Entrena un clasificador débil (generalmente un árbol de decisión de un solo nivel) con los datos ponderados.

Evaluación

Calcula el error del clasificador y su peso en el modelo final.

Actualización

Actualiza los pesos de las observaciones, aumentando los de las mal clasificadas.

Iteración

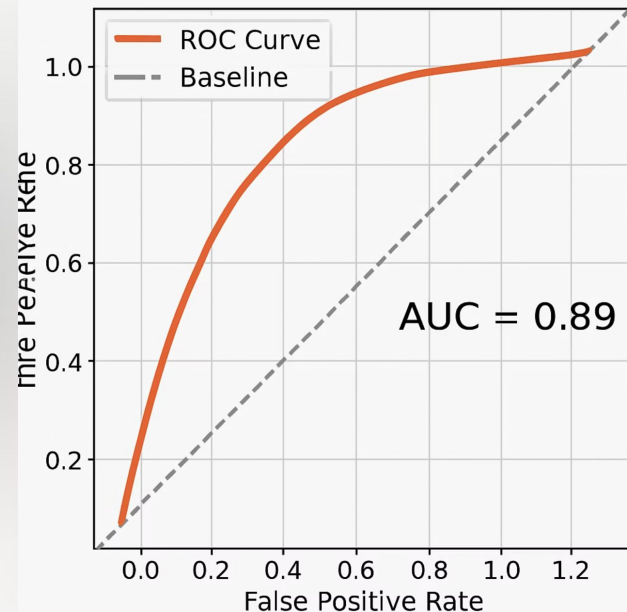
Repite los pasos 2-4 hasta alcanzar el número deseado de clasificadores.

XGBoost: La Evolución del Gradient Boosting

XGBoost (eXtreme Gradient Boosting) es una implementación optimizada del algoritmo Gradient Boosting que ha ganado popularidad por su eficiencia y rendimiento.

Desarrollado por Tianqi Chen, incorpora varias mejoras sobre el Gradient Boosting tradicional, como regularización adicional, manejo de valores faltantes y paralelización.

XGBoost



Características Principales de XGBoost

Regularización

Incluye términos de regularización L1 y L2 para controlar la complejidad del modelo y prevenir el sobreajuste.

Paralelización

Aprovecha múltiples núcleos para acelerar el entrenamiento mediante la construcción paralela de árboles.

Manejo de valores faltantes

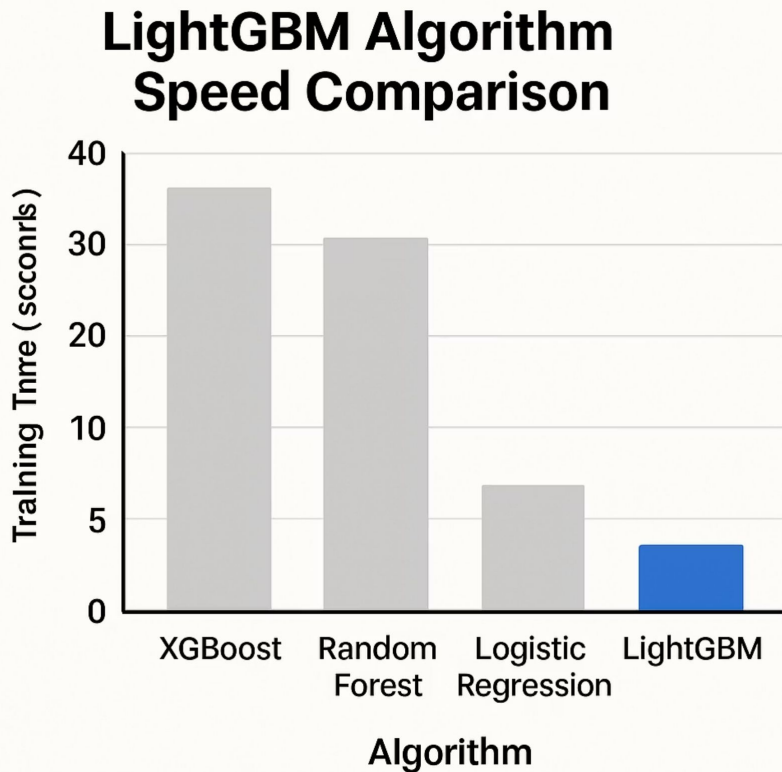
Incorpora una estrategia para tratar automáticamente los valores faltantes sin necesidad de imputación previa.

Poda de árboles

Utiliza técnicas de poda para eliminar ramas que no contribuyen significativamente a la mejora del modelo.

LightGBM: Velocidad y Eficiencia

LightGBM es otra implementación avanzada de Gradient Boosting desarrollada por Microsoft. Su principal ventaja es la velocidad de entrenamiento, especialmente en conjuntos de datos grandes. Utiliza una técnica de crecimiento de árboles basada en histogramas y una estrategia de muestreo de características que reduce significativamente el tiempo de entrenamiento.



CatBoost: Especialista en Variables Categóricas

CatBoost, desarrollado por Yandex, destaca por su manejo eficiente de variables categóricas sin necesidad de codificación previa.

Implementa un enfoque de codificación ordenada que reduce el sobreajuste y mejora la precisión en conjuntos de datos con muchas variables categóricas.

Comparativa de Implementaciones de Boosting

Algoritmo	Velocidad	Manejo de categóricas	Memoria	Regularización
Scikit-learn GB	Baja	No	Media	Básica
XGBoost	Media	No	Alta	Avanzada
LightGBM	Alta	Básico	Baja	Avanzada
CatBoost	Media	Avanzado	Media	Avanzada

Aplicaciones del Boosting en Finanzas

Detección de fraude

Los algoritmos de boosting son ampliamente utilizados para identificar transacciones fraudulentas debido a su alta precisión y capacidad para manejar datos desbalanceados.

Evaluación de riesgo crediticio

Permiten predecir la probabilidad de impago de préstamos considerando múltiples factores y sus interacciones.

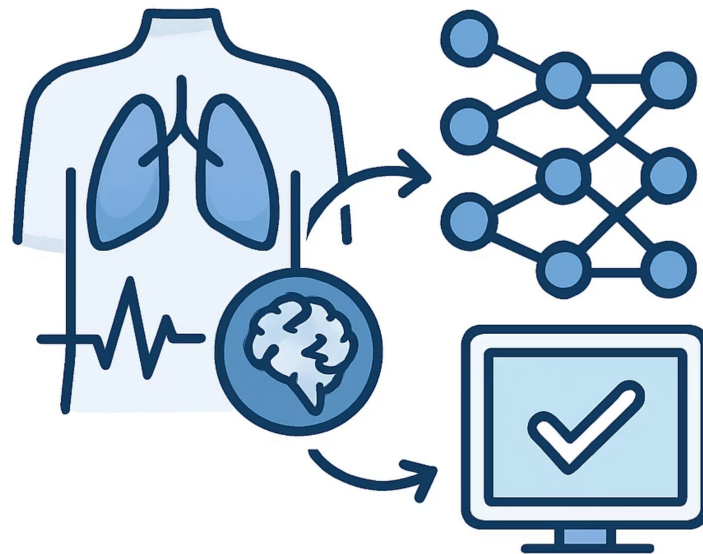
Predicción de mercados

Se utilizan para pronosticar movimientos de precios en mercados financieros, identificando patrones complejos en series temporales.

Boosting en Medicina

En el campo médico, los algoritmos de boosting han demostrado gran utilidad en:

- Diagnóstico de enfermedades a partir de imágenes médicas
- Predicción de readmisiones hospitalarias
- Identificación de factores de riesgo en pacientes
- Descubrimiento de biomarcadores para enfermedades complejas



MEDICAL DIAGNOSIS
USING MACHINE LEARNING ALGORITHMS

Boosting en Comercio Electrónico

El comercio electrónico se beneficia de los algoritmos de boosting en múltiples aspectos:



Sistemas de recomendación

Mejoran la precisión de las recomendaciones de productos basadas en el comportamiento del usuario.



Optimización de búsquedas

Refinan los resultados de búsqueda para mostrar productos más relevantes para cada usuario.



Segmentación de clientes

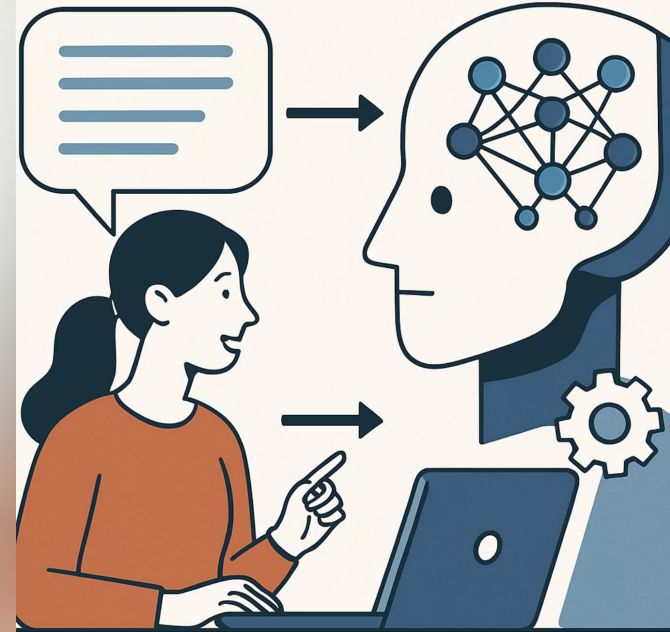
Identifican grupos de clientes con comportamientos similares para estrategias de marketing personalizadas.

Boosting en Procesamiento de Lenguaje Natural

Aunque las redes neuronales dominan actualmente el campo del NLP, los algoritmos de boosting siguen siendo relevantes en ciertas aplicaciones:

- Clasificación de textos en categorías
- Análisis de sentimiento en reseñas y comentarios
- Detección de spam en correos electrónicos
- Extracción de entidades nombradas en documentos

NATURAL LANGUAGE PROCESSING WITH MACHINE LEARNING



Regularización en Boosting

La regularización es crucial para prevenir el sobreajuste en algoritmos de boosting. Las técnicas más comunes incluyen:

1

Learning rate

Utilizar tasas de aprendizaje pequeñas (0.01-0.1) para hacer correcciones más suaves en cada iteración.

2

Subsampling

Entrenar cada árbol con solo una fracción aleatoria de los datos (similar al bagging).

3

Limitación de profundidad

Restringir la profundidad máxima de los árboles para evitar que capturen ruido.

4

Early stopping

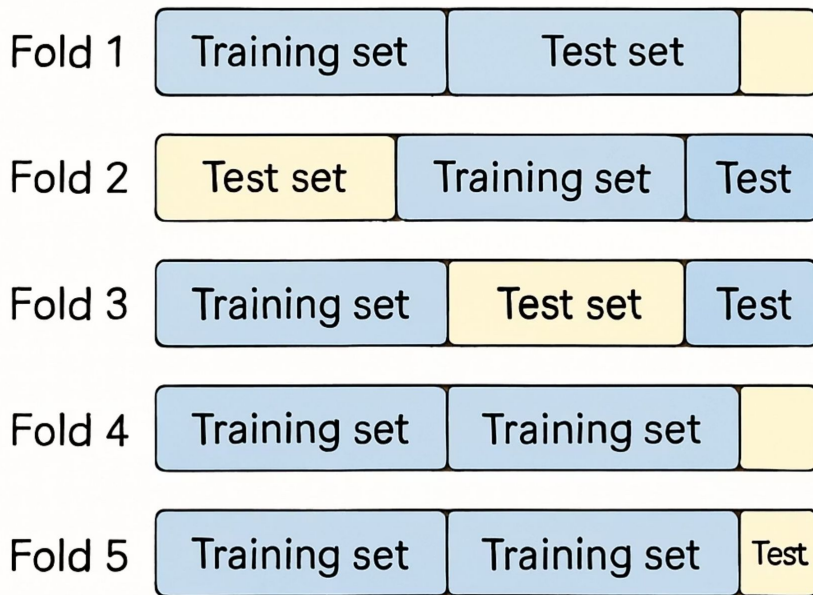
Detener el entrenamiento cuando el rendimiento en un conjunto de validación deja de mejorar.

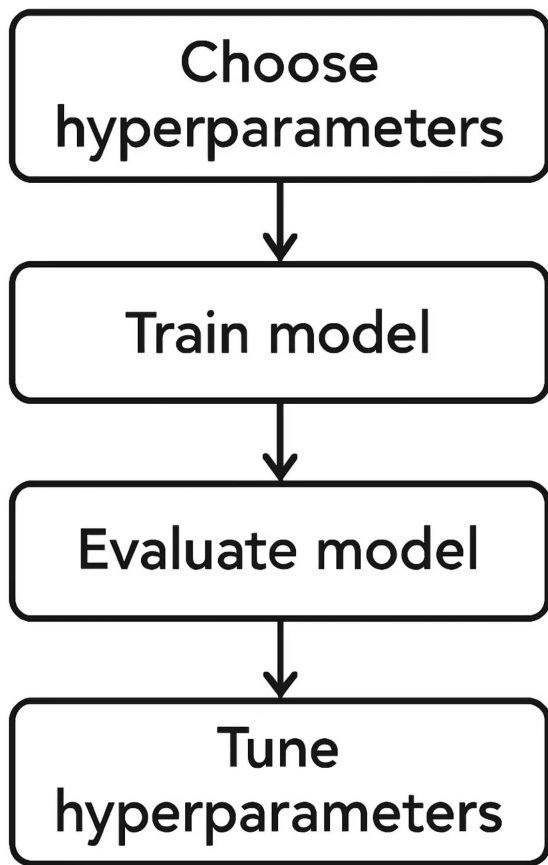
Validación Cruzada en Boosting

La validación cruzada es esencial para evaluar correctamente el rendimiento de los modelos de boosting y evitar el sobreajuste. Las estrategias más comunes son:

- K-fold cross-validation
- Leave-one-out cross-validation
- Stratified cross-validation para datos desbalanceados

Cross-Validation





Ajuste de Hiperparámetros

El rendimiento óptimo de los algoritmos de boosting depende en gran medida del ajuste adecuado de sus hiperparámetros. Las técnicas más utilizadas para este fin son:

Grid Search

Evalúa todas las combinaciones posibles de un conjunto predefinido de valores para cada hiperparámetro.

Random Search

Prueba combinaciones aleatorias dentro de rangos especificados, siendo más eficiente que Grid Search para espacios grandes.

Bayesian Optimization

Utiliza modelos probabilísticos para dirigir la búsqueda hacia regiones prometedoras del espacio de hiperparámetros.

Interpretabilidad de Modelos de Boosting

Aunque los modelos de boosting son considerados "cajas negras", existen técnicas para interpretar sus predicciones:

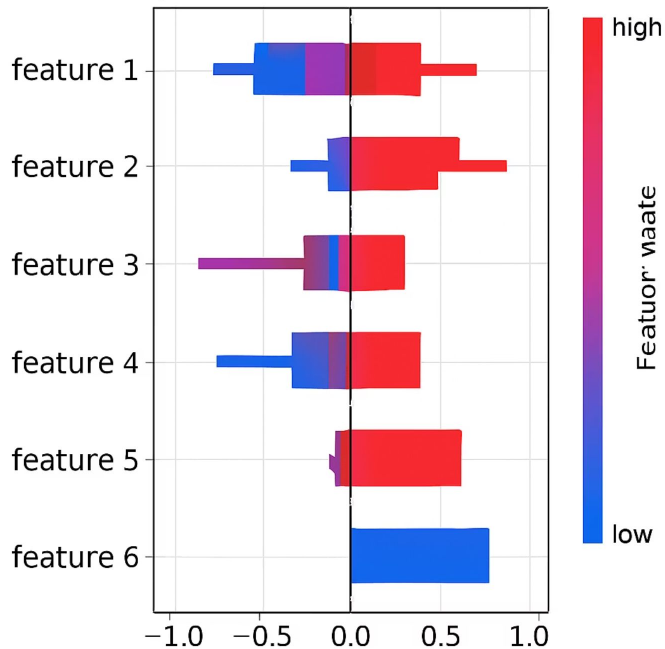
Importancia de características

Mide cuánto contribuye cada variable a la mejora del modelo, identificando las más relevantes para la predicción.

Gráficos de dependencia parcial

Muestran cómo cambia la predicción al variar una característica mientras se mantienen constantes las demás.

SHAP values (impact on model output)



SHAP Values

Los valores SHAP (SHapley Additive exPlanations) representan una técnica avanzada para interpretar modelos complejos como los de boosting.

Basados en la teoría de juegos cooperativos, asignan a cada característica una contribución a la predicción final para cada instancia individual, permitiendo explicaciones locales y globales del modelo.

Boosting vs. Redes Neuronales

Aspecto	Boosting	Redes Neuronales
Datos tabulares	Excelente	Bueno
Imágenes/audio	Limitado	Excelente
Texto	Bueno	Excelente
Interpretabilidad	Media	Baja
Tiempo de entrenamiento	Medio	Alto
Datos necesarios	Menos	Más

Cada enfoque tiene sus fortalezas y el contexto del problema determina cuál es más adecuado.

Boosting en Competencias de Ciencia de Datos

Los algoritmos de boosting, especialmente XGBoost y LightGBM, han dominado las competencias de ciencia de datos como Kaggle durante años.

Su popularidad se debe a su capacidad para producir modelos precisos con relativamente poco preprocesamiento de datos y ajuste de hiperparámetros.

Factores de Éxito en Competencias

1

Ensamblado de modelos

Combinar múltiples algoritmos de boosting con diferentes configuraciones para mejorar aún más la precisión.

2

Ingeniería de características

Crear nuevas variables derivadas que capturen relaciones importantes en los datos.

3

Validación robusta

Diseñar esquemas de validación que reflejen fielmente la evaluación final del modelo.

4

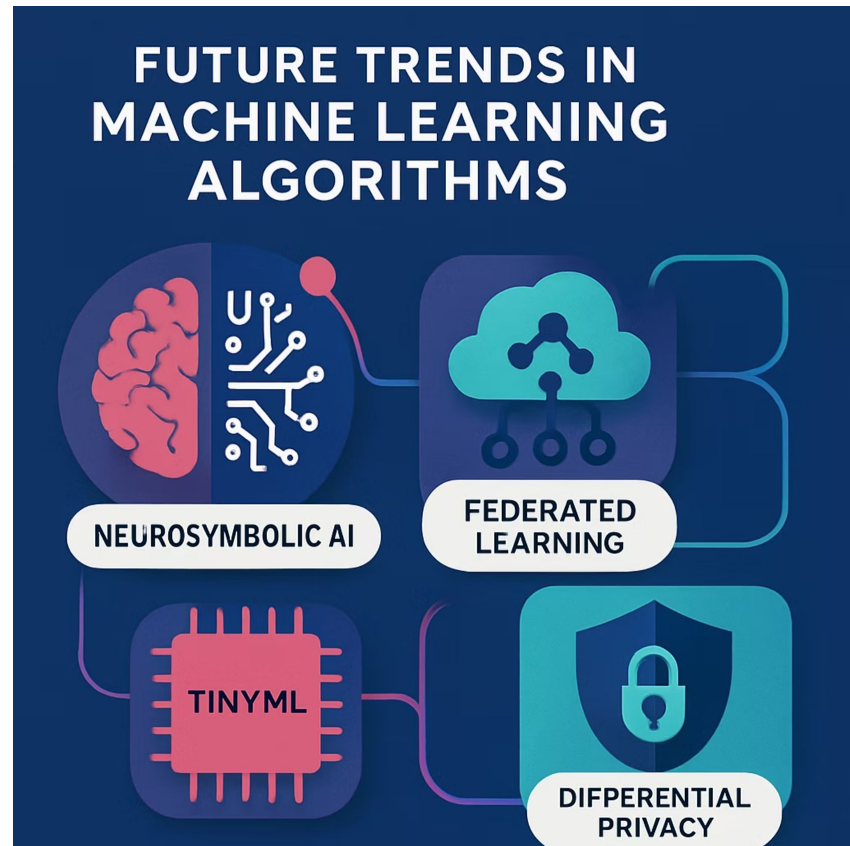
Ajuste fino

Dedicar tiempo a optimizar meticulosamente los hiperparámetros para el problema específico.

Tendencias Futuras en Boosting

El campo de los algoritmos de boosting continúa evolucionando con nuevas direcciones de investigación:

- Integración con técnicas de aprendizaje profundo
- Mejoras en la eficiencia computacional
- Adaptación para problemas de aprendizaje por refuerzo
- Desarrollo de variantes para aprendizaje federado y privacidad diferencial



Boosting en Sistemas de Producción

Implementar modelos de boosting en entornos de producción requiere consideraciones específicas:

Serialización de modelos

Guardar y cargar modelos eficientemente para su despliegue.

Monitoreo de rendimiento

Vigilar la degradación del modelo con el tiempo debido a cambios en los datos.

Latencia de predicción

Optimizar el tiempo de respuesta para aplicaciones en tiempo real.

Reentrenamiento periódico

Establecer procesos para actualizar modelos con nuevos datos.

Herramientas para Despliegue



MLflow

Plataforma para gestionar el ciclo de vida completo de los modelos de ML, incluyendo experimentación, reproducibilidad y despliegue.



Servicios cloud

AWS SageMaker, Google AI Platform y Azure ML ofrecen soluciones integradas para desplegar modelos de boosting a escala.



Contenedores

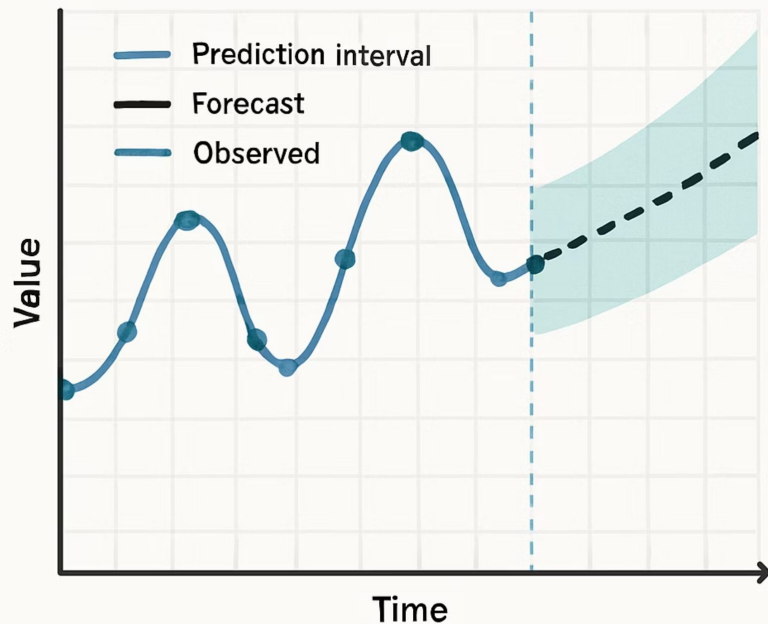
Docker y Kubernetes permiten empaquetar modelos con sus dependencias para un despliegue consistente en diferentes entornos.

Boosting para Series Temporales

Los algoritmos de boosting también pueden adaptarse para problemas de series temporales:

- Incorporación de características temporales (lag, tendencia, estacionalidad)
- Validación específica para datos secuenciales (rolling window)
- Combinación con modelos específicos de series temporales (ARIMA, Prophet)

TIME SERIES FORECASTING WITH MACHINE LEARNING



Boosting para Datos Desbalanceados

Los problemas con clases desbalanceadas son comunes en aplicaciones reales, y los algoritmos de boosting pueden adaptarse para manejarlos:

Ponderación de clases

Asignar mayor peso a las clases minoritarias durante el entrenamiento.

Funciones de pérdida específicas

Utilizar métricas como F1-score o AUC-ROC que son más adecuadas para datos desbalanceados.

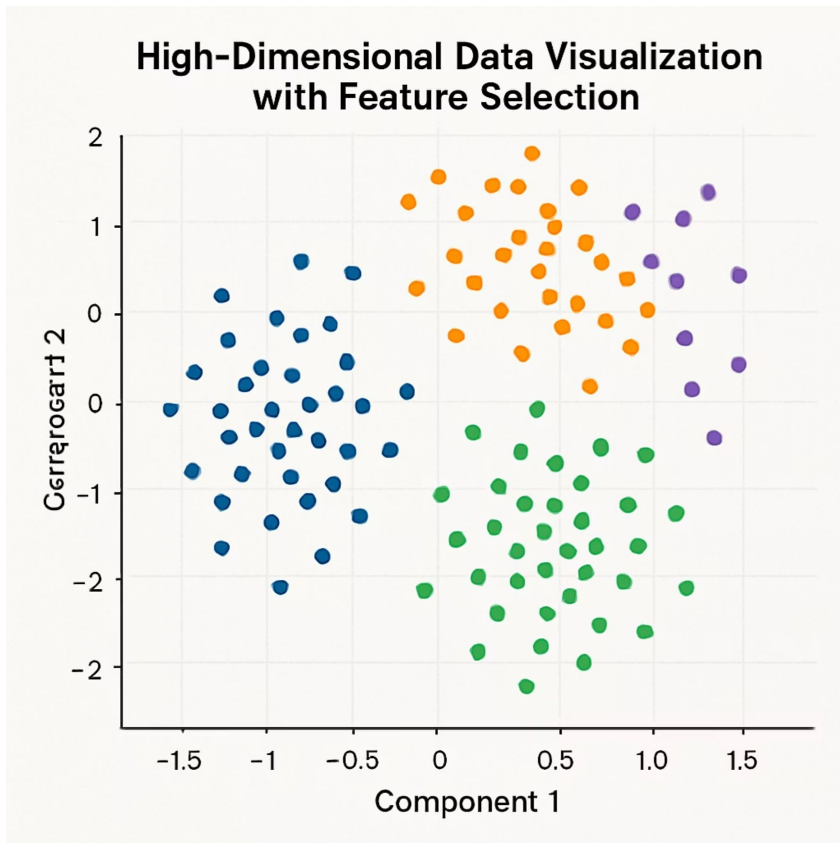
Combinación con técnicas de muestreo

Integrar SMOTE u otras técnicas de sobremuestreo/submuestreo con boosting.

Boosting para Datos de Alta Dimensionalidad

Cuando se trabaja con conjuntos de datos con muchas características, los algoritmos de boosting ofrecen ventajas específicas:

- Selección implícita de características al construir árboles
- Robustez frente a características irrelevantes
- Capacidad para capturar interacciones complejas entre variables



Recursos de Aprendizaje

Para profundizar en el estudio de los algoritmos de boosting, existen numerosos recursos disponibles:



Libros

- "The Elements of Statistical Learning" de Hastie, Tibshirani y Friedman
- "Hands-On Gradient Boosting with XGBoost and scikit-learn" de Packt Publishing



Cursos en línea

- Coursera: "How to Win a Data Science Competition"
- Kaggle Learn: "Intermediate Machine Learning"



Documentación

- Documentación oficial de XGBoost, LightGBM y CatBoost
- Tutoriales de Scikit-learn sobre ensemble methods

DATA SCIENCE COMMUNITY COLLABORATION



Comunidades y Foros

Participar en comunidades especializadas puede acelerar el aprendizaje y resolver dudas específicas:



Kaggle

Foros de discusión, kernels compartidos y competencias para practicar.



Stack Overflow

Preguntas y respuestas sobre implementación y problemas técnicos.



GitHub

Repositorios de código, issues y discusiones sobre las bibliotecas.

Resumen de Conceptos Clave

Fundamentos

- Boosting combina modelos débiles secuencialmente
- Cada modelo corrige errores del anterior
- Reduce principalmente el sesgo del modelo

Algoritmos

- AdaBoost: ajusta pesos de observaciones
- Gradient Boosting: minimiza función de pérdida
- XGBoost/LightGBM/CatBoost: implementaciones optimizadas

Prácticas

- Regularización para evitar sobreajuste
- Validación cruzada para evaluación robusta
- Ajuste de hiperparámetros para optimizar rendimiento

Conclusión Final

Los algoritmos de boosting representan una de las técnicas más potentes y versátiles en el campo del aprendizaje automático. Su capacidad para combinar modelos débiles en un predictor fuerte los hace especialmente valiosos en problemas complejos donde otros enfoques fallan. Aunque presentan desafíos como el tiempo de entrenamiento y el riesgo de sobreajuste, las implementaciones modernas y las técnicas de regularización han mitigado en gran medida estas limitaciones.

Dominar estos algoritmos proporciona una herramienta fundamental para cualquier científico de datos, permitiendo construir modelos precisos y robustos para una amplia gama de aplicaciones. El futuro del boosting parece prometedor, con continuas innovaciones que expanden sus capacidades y eficiencia, manteniéndolo relevante incluso en la era del aprendizaje profundo.

Live Coding

¿En qué consistirá la Demo?

Vamos a implementar modelos avanzados de boosting con XGBoost y CatBoost, comparando su rendimiento con GradientBoostingClassifier. También exploraremos interpretabilidad con SHAP.

1. Cargar el dataset Breast Cancer.
2. Entrenar un modelo con XGBClassifier.
3. Ajustar parámetros: max_depth, learning_rate, n_estimators.
4. Evaluar métricas: accuracy, F1, AUC.
5. Repetir con CatBoostClassifier.
6. Visualizar importancia de variables.
7. Aplicar SHAP para explicación local y global.
8. Comparar rendimiento y tiempos.



Momento:

Time-out!



5 -10 min.





Ejercicio

Comparativa entre Boosting Avanzado

Comparativa entre Boosting Avanzado

Contexto: 🙌

Una fintech desea implementar un sistema de scoring crediticio para evaluar la probabilidad de impago de nuevos clientes. Necesitan un modelo que sea altamente preciso, interpretable y eficiente para producción.

Consigna: 📝

1. Utilizar el dataset German Credit Data (lo podés cargar con `openml.datasets`).
2. Realizar preprocesamiento: manejo de categóricas y escalamiento.
3. Entrenar un modelo con `XGBoostClassifier`.
4. Repetir con `CatBoostClassifier`.
5. Comparar métricas: accuracy, F1-score, curva ROC.
6. Evaluar tiempos de entrenamiento y facilidad de uso.
7. Generar visualizaciones de importancia de variables.
8. Aplicar SHAP para interpretar una predicción individual.
9. Concluir cuál modelo es más conveniente para producción.

Paso a paso: ⚙️

1. Cargar y procesar los datos
2. Modelado con XGBoost y CatBoost
3. Comparar resultados
4. Aplicar SHAP
5. Redactar recomendaciones

Tiempo 🕒: 45 Minutos

¿Alguna consulta?





Resumen

¿Qué logramos en esta clase?



- ✓ Implementamos AdaBoost, XGBoost, LightGBM y CatBoost
- ✓ Ajustamos hiperparámetros con distintos métodos
- ✓ Exploramos técnicas de regularización en boosting
- ✓ Evaluamos interpretabilidad con SHAP
- ✓ Vimos aplicaciones reales en múltiples industrias



¡Ponte a prueba!

Momento de ejercitación

Te invitamos a aprovechar esta última sección del espacio sincrónico para realizar de manera individual las **actividades disponibles en la plataforma**. Estas propuestas son clave para afianzar lo trabajado y **forman parte obligatoria del recorrido de aprendizaje**.

👉 **Análisis de caso** ————— 👉 **Selección Múltiple**

👉 **Comprensión lectora**

Si al resolverlas surge alguna duda, compartela o tráela al próximo encuentro sincrónico.

< **¡Muchas gracias!** >

